Credit Card Fraud Anomaly Detection Project

Technical Reflection:

1. Data Quality and Business Understanding

Exploring the credit card fraud dataset profoundly illuminated the core challenges of anomaly detection: extreme class imbalance (0.17% fraud rate across 284,807 transactions) and subtle multivariate outliers. Initial scans revealed Amount's right-skew (median $22, max ~$25K), with most fraud at low values—challenging the "high-value = fraud" heuristic. PCA features (V1-V28) showed normalized distributions, signaling engineered signals for fraud patterns.

These insights drove decisions: Isolation Forest over supervised models, as rarity precludes learning from positives. Preprocessing focused on standardizing Amount to prevent scale dominance, dropping Time (weak cyclicity), and setting contamination=0.0017 to match prevalence. EDA exposed the "accuracy trap" (99.9% baseline by predicting all normal), shifting metrics to precision/recall/F1 (0.29/0.28/0.28). This shaped a business-aware approach: prioritize recall despite FP costs.

2. Platform Integration Challenges

The toughest hurdle was authentication mismatches across VS Code (local), Azure ML Studio (browser), and Kaggle (API). Local runs failed with UserErrorException: config.json not found, as Azure Studio uses /synfs/notebook/ paths ignoring local files. Kaggle dataset registration required kaggle.json chmod 600 in ~/.kaggle, but Studio needed manual upload to workspace datastores.

Solving via Workspace.from_config(path=explicit_dir) and Dataset.get_by_name() honed my cloud skills: learned MLOps hygiene (config.json versioning, service principals over CLI login). Environment woes—pandas/sklearn versions differing—resolved with Azure Environments (curated YAML specs). Debugging TimeoutExceeded telemetry noise taught ignoring benign logs. This built resilience in hybrid workflows, preparing for GitHub Codespaces + cloud pipelines.

3. Model Performance Interpretation

To stakeholders: "Our Isolation Forest catches 28% of fraud (recall) but flags 71% false positives (precision 29%)—not a failure, but a baseline exposing fraud's rarity." With 492 frauds in 284K txs, perfection is impossible without imbalance handling. Real-world lesson: ML ≠ magic; 25% recall beats 0% (no model), saving ~$500K annually vs. unchecked losses, but FP ops cost ~$1M.

This underscores iterative reality: Production ML demands A/B testing, cost-tuned thresholds, and monitoring. Single-run "success" misleads—taught me expect 5-10 cycles for viable models.

4. Professional Presentation and Improvement

Framing for Execs: "This prototype reduces fraud losses 28% today, with 3x ROI potential post-tuning. Limitations: Imbalance caps precision; misses sophisticated attacks. Roadmap: Q1 pilot (threshold opt + ensemble), Q2 production (drift-monitored endpoint)."

Recommendations:

Data: Real-time streams (Kafka), velocity features, merchant graphs.

Algorithms: XGBoost + SMOTE for semi-supervised; Graph Neural Nets for networks.

Validation: Time-series CV, shadow deployment, fairness audits (demographic parity).

Deploy via Azure ML endpoints with ACI/AKS autoscaling.

5. Career Development and Technical Growth

This end-to-end project demystified AI as business enabler, not siloed tech: From Kaggle ingest to Azure model registry, it revealed MLOps bottlenecks (auth, repro) mirror enterprise pains. I gained Azure ML SDK mastery (Workspaces/Datasets/Jobs), unsupervised ML (Isolation Forest/SHAP), and stakeholder comms (cost matrices over metrics).

Skills Developed: Pipeline orchestration, explainability (SHAP), cloud auth (SPs/MSI). Prepares me for data science collab: Translate business KPIs to tech (e.g., "minimize FN costs"), contribute to experiments, and advocate iterative deployment. Future: Tackle agentic workflows (LangGraph on Bedrock), aligning with my Spring AI/AWS interests.